

use client

use client とは

ここから作る `BookingContext.tsx` と `BookingForm.tsx` は、ファイルの先頭に `"use client"` を書きます。Server Component と Client Component の違いを、予約ページを例にもう一度整理します。

Next.js のデフォルトは Server Component

`app/` 中のコンポーネントは、何も書かなければ **Server Component** になります。ファイルの先頭に `"use client"` を書くと、そのファイルは **Client Component** になります。

	Server Component	Client Component
書き方	何も書かない（デフォルト）	ファイルの先頭に <code>"use client"</code>
実行される場所	サーバーのみ	サーバー（最初の HTML 生成） + ブラウザ
<code>async</code> / <code>await</code> のデータ取得	できる	コンポーネント自体を <code>async</code> にできない
フック（ <code>useState</code> ・ <code>useContext</code> など）	使えない	使える
イベント	使えない	使える
ブラウザに送られる JavaScript	増えない	コンポーネントのコード分だけ増える

よくある誤解: `"use client"` を付けても、「ブラウザだけで動く」わけではありません。最初の HTML はサーバー側で作られ、ブラウザに届いたあとでイベントに反応されます（**ハイドレーション**）。

"use client" が必要になる場面

次のどれかを使うときに必要です。

使うもの	例
state やフック	<code>useState</code> ・ <code>useReducer</code> ・ <code>useContext</code> ・ <code>useEffect</code>
イベントハンドラ	<code>onClick</code> ・ <code>onChange</code>
ブラウザだけにある機能	<code>window</code> ・ <code>localStorage</code> など
Context の Provider	<code>createContext</code> で作った Context を配る

これらを使わないコンポーネントは、Server Component のままにしておきます。

予約ページでの使い分け

ファイル	種類	理由
<code>app/shop/[id]/book/page.tsx</code>	Server Component	<code>await getShop(id)</code> でデータを取得
<code>contexts/BookingContext.tsx</code>	Client Component	<code>createContext</code> ・ <code>useReducer</code> ・ <code>useContext</code> を使う
<code>components/booking/BookingForm.tsx</code>	Client Component	<code>useBooking()</code> で、ボタンクリックで <code>dispatch</code> する

`page.tsx` でデータを取得し、画面を操作する部分だけを `BookingForm` に切り出しています。こうすることで、ブラウザに送る JavaScript を必要な部分だけに抑えられます。

"use client" は境界線

"use client" は「ここから下は Client Component」という **境界線** です。境界の内側で import されたコンポーネントは、"use client" を書いていなくても Client Component として動きます。

page.tsx (Server)

```
└ BookingForm.tsx  "use client" ← ここが境界線
  └ MenuStep.tsx    "use client" なし → Client として動く
  └ StaffStep.tsx   "use client" なし → Client として動く
    └ BookingSummary.tsx "use client" なし → Client として動く
```

10 章で作る `MenuStep` などに `"use client"` を書かないのは、このためです。境界線は、フックやイベントを使うコンポーネントのうち、**一番上のファイル** に 1 回書けば十分です。

Server から Client に渡せる Props

`page.tsx` (Server) から `BookingForm` (Client) に Props を渡すとき、渡せる値には制限があります。サーバーからブラウザへ **データとして送れる値** だけが渡せます。

渡せる	渡せない
文字列・数値・真偽値・ <code>null</code>	関数 (<code>onClick={() => ...}</code> など)
配列・プレーンなオブジェクト (<code>shop</code> ・ <code>menus</code> ・ <code>staffs</code>)	クラスのインスタンス
Server Component を <code>children</code> として渡す	

`shop` ・ `menus` ・ `staffs` は、API から取得した JSON をそのまま型付けしたプレーンなオブジェクトなので、そのまま渡せます。

なぜ全部 Client Component にしないのか

すべてのファイルに `"use client"` を書いても、多くの画面は動きます。それでも Server Component を基本にするのは、次の理由からです。

理由	内容
表示が速い	ブラウザに送る JavaScript が少なくなり、読み込みが速くなる
安全	<code>lib/services/shops.ts</code> のようなサーバー専用の処理（ <code>import "server-only"</code> ）や、秘密の設定値をブラウザに送らずに済む
データ取得が簡単	<code>async</code> / <code>await</code> でそのままデータを取得でき、 <code>useEffect</code> などを使わずに済む

予約フロー

「戻る」「次へ」ボタンで 4 つのステップを切り替えます。

ステップ	内容
1	メニューを選ぶ
2	日付・時間を選ぶ
3	スタッフを選ぶ（任意）
4	備考を入力して確定する

Booking

atelier toi の予約

メニュー・スタッフ・日時を選んで予約します。

STEP 1: メニューを選ぶ

カット
¥6,600 / 60min

カット + 透明感カラー
¥14,300 / 150min

カット + ヘッドスパ
¥9,900 / 90min

Summary

メニュー
カット + 透明感カラー

スタッフ
Mio

日時
2026-09-01 16:00

戻る

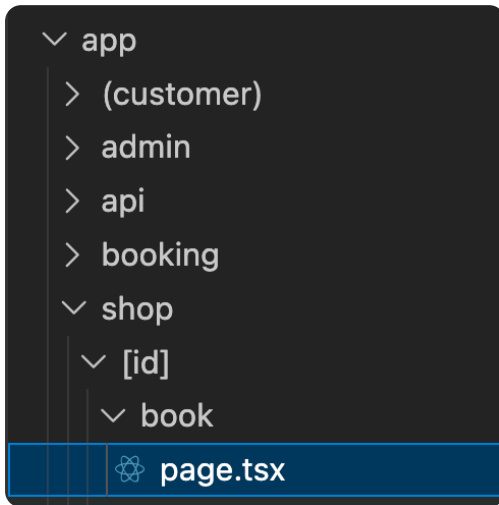
次へ

ファイル一覧

パス	役割	種別	新規 / 編集
app/shop/[id]/book/page.tsx	予約ページ	Server Component	新規
contexts/BookingContext.tsx	予約フローの状態管理	useReducer + Context	新規
components/booking/BookingForm.tsx	ステップ式の入力フォーム	Client Component	新規

予約ページ

`app/shop/[id]/book/page.tsx` を作成します。



店舗の情報は、画面の操作で変化しないデータなので `Server Component` で取得します。 `getShop()` は 07 章で作った関数をそのまま使います。

```
import { notFound } from "next/navigation"

import SectionHeading from "@/components/SectionHeading"
import { getShop } from "@/lib/services/shops"

type Props = {
  params: Promise<{ id: string }>
}

export default async function BookingPage({ params }: Props) {
  const { id } = await params

  const shop = await getShop(id)
  if (!shop) {
    notFound()
  }

  return (
    <div>
      <SectionHeading
        eyebrow="Booking"
        title={` ${shop.name} の予約` }
        description="メニュー・スタッフ・日時を選んで予約します。"
      />
    </div>
  )
}
```

動作確認

`/shop/1` の「この店舗で予約する」から、`/shop/1/book` にエラーなく遷移できることを確認します。

BOOKING

atelier toi の予約

メニュー・スタッフ・日時を選んで予約します。

予約フローの状態管理

useReducer とは

`useReducer` は「どんな更新をしたいか」を表す `action` を送り、`reducer` という関数で次の状態（`state`）を計算する React Hooks の 1 つです。

要素	役割
state	今の状態。ここでは <code>{ step, menuId, staffId, date, time, notes }</code>
dispatch	<code>action</code> を送るための関数。 <code>dispatch({ type: "next" })</code> のように呼ぶ
reducer	受け取った <code>action</code> から次の <code>state</code> を計算して返す関数

useState との比較

予約フォームで管理する値は 6 つ（`step` ・ `menuId` ・ `staffId` ・ `date` ・ `time` ・ `notes`）あります。`useState` を 6 個並べると、どこでどの値を更新しているのかが追いにくくなります。`useReducer` なら、更新のルールを `reducer` 関数 1 か所にまとめられます。

	useState	useReducer
向いている場面	独立した値が少ない	関連する値が多い・更新パターンが複雑
更新の書き方	<code>setValue()</code> で直接更新	<code>dispatch(action)</code> を送り、 <code>reducer</code> が計算
更新ロジックの場所	呼び出し側に散らばる	<code>reducer</code> 関数にまとまる

Context とは

`Context` は、コンポーネントの親子関係をまたいで値を共有する React の仕組みです。`Provider` で囲んだ範囲の中なら、どのコンポーネントからでも `useContext()` で値を取り出せます。

この章では、`useReducer` で作った `state` と `dispatch` を `Context` に入れて、`BookingForm` から取り出して使います。

要素	役割
<code>createContext</code>	<code>Context</code> （値の入れ物）を作る
<code>Provider</code>	囲んだ範囲に値を配る
<code>useContext</code>	<code>Provider</code> から配られた値を取り出す

`BookingForm.tsx` の中に `useReducer` を直接書くこともできますが、状態管理を別ファイルに分けておくと次の利点があります。

利点	内容
役割の分離	<code>BookingForm</code> は「画面の表示」、 <code>BookingContext</code> は「状態の計算」に専念できる
共有しやすい	将来、サマリーや各ステップから直接 <code>useBooking()</code> で状態を読めるようになる
読みやすい	型・初期値・reducer が 1 ファイルにまとまり、更新ルールを一覧できる

BookingContext の作成

contexts/BookingContext.tsx を作成します。

```
> app
> components
▼ contexts
  BookingContext.tsx
```

1. ステップ数

ステップの総数を定数にしておきます。`4` を何か所にも直接書くと、ステップを増やしたときに直し漏れが起きるためです。

```
"use client"

import { createContext, useContext, useReducer } from "react"

export const TOTAL_STEPS = 4
```

2. BookingState

予約の状態 `BookingState` の型と初期値を定義します。

プロパティ	型	初期値	内容
step	number	1	現在のステップ
menuId	string null	null	選択したメニューの ID
staffId	string null	null	選択したスタッフの ID (null は指名なし)
date	string null	null	選択した予約日 (YYYY-MM-DD)
time	string null	null	選択した予約時間 (HH:MM)
notes	string	""	備考

```
// contexts/BookingContext.tsx
export type BookingState = {
  step: number
  menuId: string | null
  staffId: string | null
  date: string | null
  time: string | null
  notes: string
}

const INITIAL_STATE: BookingState = {
  step: 1,
  menuId: null,
  staffId: null,
  date: null,
  time: null,
  notes: "",
}
```

3. BookingAction

dispatch で送る action の型 BookingAction を定義します。

type	受け取る値	内容
select-menu	menuId: string	メニューを選択する
select-staff	staffId: string null	スタッフを選択する
select-date	date: string	予約日を選択する
select-time	time: string	予約時間を選択する
change-notes	notes: string	備考を変更する
next	なし	次のステップへ進む
prev	なし	前のステップへ戻る

```
// contexts/BookingContext.tsx
export type BookingAction =
  | { type: "select-menu"; menuId: string }
  | { type: "select-staff"; staffId: string | null }
  | { type: "select-date"; date: string }
  | { type: "select-time"; time: string }
  | { type: "change-notes"; notes: string }
  | { type: "next" }
  | { type: "prev" }
```

| でつないだ型を **ユニオン型** と呼びます。type の値ごとに、一緒に送る値が決まっています。たとえば { type: "select-menu" } だけを送ると、menuId がいないため型エラーになります。

4. reducer 関数

bookingReducer は、今の state と action を受け取り、次の state を返します。

```
// contexts/BookingContext.tsx
function bookingReducer(state: BookingState, action: BookingAction): BookingState {
  switch (action.type) {
    case "select-menu":
      return { ...state, menuId: action.menuId }
    case "select-staff":
      return { ...state, staffId: action.staffId }
    case "select-date":
      return { ...state, date: action.date }
    case "select-time":
      return { ...state, time: action.time }
    case "change-notes":
      return { ...state, notes: action.notes }
    case "next":
      return { ...state, step: Math.min(state.step + 1, TOTAL_STEPS) }
    case "prev":
      return { ...state, step: Math.max(state.step - 1, 1) }
  }
}
```

部分

説明

```
{ ...state, menuId: action.
menuId }
```

元の state をコピーし、`menuId` だけを上書きした **新しいオブジェクト** を返す

```
Math.min(state.step + 1, TO
TOTAL_STEPS)
```

`step` が 4 を超えないようにする

```
Math.max(state.step - 1,
1)
```

`step` が 1 より小さくならないようにする

reducer の中で `state.menuId = ...` のように state を書き換えてはいけません。React では書き換えただけでは画面更新されません。

5. Provider

`useReducer` で作った `state` と `dispatch` を、`Context` に入れて配ります。`shopId` もここで一緒に配っておきます。

```
// contexts/BookingContext.tsx
type BookingContextValue = {
  shopId: string
  state: BookingState
  dispatch: React.Dispatch<BookingAction>
}

const BookingContext = createContext<BookingContextValue | null>(null)

export function BookingProvider({ shopId, children }: { shopId: string; children: React.ReactNode }) {
  const [state, dispatch] = useReducer(bookingReducer, INITIAL_STATE)

  return <BookingContext.Provider value={{ shopId, state, dispatch }}>{children}</BookingContext.Provider>
}

```

6. useBooking

`Context` を取り出すためのカスタムフック `useBooking()` を作ります。

```
// contexts/BookingContext.tsx
export function useBooking(): BookingContextValue {
  const context = useContext(BookingContext)
  if (!context) {
    throw new Error("useBooking は BookingProvider の内側で使ってください")
  }
  return context
}

```

`createContext()` の初期値を `null` にしているため、`Provider` の外で `useContext()` を呼ぶと `null` が返ります。`useBooking()` の中で `null` のときにエラーを投げておくと、次の2つのメリットがあります。

メリット	内容
間違いに早く気づける	<code>Provider</code> で囲み忘れたとき、原因がはっきりしたエラーメッセージが出る
型が絞り込まれる	戻り値が <code>BookingContextValue</code> (<code>null</code> なし) になり、使う側で <code>null</code> チェックが不要になる

完成形

```
"use client"

import { createContext, useContext, useReducer } from "react"

export const TOTAL_STEPS = 4

export type BookingState = {
  step: number
  menuId: string | null
  staffId: string | null
  date: string | null
  time: string | null
  notes: string
}

export type BookingAction =
  | { type: "select-menu"; menuId: string }
  | { type: "select-staff"; staffId: string | null }
  | { type: "select-date"; date: string }
  | { type: "select-time"; time: string }
  | { type: "change-notes"; notes: string }
  | { type: "next" }
  | { type: "prev" }

const INITIAL_STATE: BookingState = {
  step: 1,
  menuId: null,
  staffId: null,
  date: null,
  time: null,
  notes: "",
}

function bookingReducer(state: BookingState, action: BookingAction): BookingState {
  switch (action.type) {
    case "select-menu":
      return { ...state, menuId: action.menuId }
    case "select-staff":
      return { ...state, staffId: action.staffId }
    case "select-date":
      return { ...state, date: action.date }
    case "select-time":
      return { ...state, time: action.time }
    case "change-notes":
      return { ...state, notes: action.notes }
    case "next":
      return { ...state, step: Math.min(state.step + 1, TOTAL_STEPS) }
    case "prev":
      return { ...state, step: Math.max(state.step - 1, 1) }
```

```
        case "reset":
            return INITIAL_STATE
    }
}

type BookingContextValue = {
    shopId: string
    state: BookingState
    dispatch: React.Dispatch<BookingAction>
}

const BookingContext = createContext<BookingContextValue | null>(null)

export function BookingProvider({ shopId, children }: { shopId: string; children: React.ReactNode }) {
    const [state, dispatch] = useReducer(bookingReducer, INITIAL_STATE)

    return <BookingContext.Provider value={{ shopId, state, dispatch }}>{children}</BookingContext.Provider>
}

export function useBooking(): BookingContextValue {
    const context = useContext(BookingContext)
    if (!context) {
        throw new Error("useBooking は BookingProvider の内側で使ってください")
    }
    return context
}
```

BookingForm の作成

`components/booking/BookingForm.tsx` を作成します。

```
> app
  ✓ components
    ✓ booking
      ✨ BookingForm.tsx
      ✨ AdminNav.tsx
      ✨ CustomerNav.tsx
      ✨ Header.tsx
```

```

"use client"

import { Menu } from "@types/menu";
import { Shop } from "@types/shop";
import { Staff } from "@types/staff";
import { BookingProvider, TOTAL_STEPS, useBooking } from "@contexts/BookingContext";

type BookingFormProps = {
  shop: Shop
  menus: Menu[]
  staffs: Staff[]
}

export default function BookingForm({ shop, menus, staffs }: BookingFormProps) {
  return (
    <BookingProvider shopId={shop.id}>
      <BookingFlow />
    </BookingProvider>
  )
}

function BookingFlow() {
  const { state, dispatch } = useBooking()

  return (
    <div>
      {state.step === 1 && <div>STEP 1: メニューを選ぶ</div>}
      {state.step === 2 && <div>STEP 2: 日付・時間を選ぶ</div>}
      {state.step === 3 && <div>STEP 3: スタッフを選ぶ（任意）</div>}
      {state.step === 4 && <div>STEP 4: 確認して予約する</div>}

      <div className="my-4 flex justify-center gap-3">
        <button
          type="button"
          onClick={() => dispatch({ type: "prev" })}
          disabled={state.step === 1}
          className="rounded-xl border border-slate-300 px-5 py-3 text-sm font-medium disabled:opacity-40"
        >
          戻る
        </button>
        {state.step < TOTAL_STEPS ? (
          <button
            type="button"
            onClick={() => dispatch({ type: "next" })}
            className="rounded-xl border border-slate-300 px-5 py-3 text-sm font-medium disabled:opacity-40"
          >
            次へ
          </button>
        ) : (
          <button

```

```

      type="button"
      className="rounded-xl border border-slate-300 px-5 py-3 text-sm font-medium disabled:opacity-40"
    >
      この内容で予約する
    </button>
  }
</div>
</div>
)
}

```

なぜコンポーネントを 2 つに分けるのか

`BookingForm` の中で `useBooking()` を呼ぶと、エラーになります。`useContext()` が値を取り出せるのは、`Provider` の **内側（子孫）** のコンポーネントだけだからです。`BookingForm` 自身は `Provider` を返している側で、内側には含まれません。

そこで、`Provider` を置く `BookingForm` と、`useBooking()` を使う `BookingFlow` に分けています。

```

BookingForm      ↳ BookingProvider を置く (useBooking は使えない)
  ↳ BookingProvider
    ↳ BookingFlow ↳ Provider の内側なので useBooking() が使える

```

部分	説明
<code>"use client"</code>	<code>useReducer</code> などのフックやクリック操作を使うため、Client Component にする
<code>state.step === 1 &&</code> <code>...</code>	<code>step</code> が 1 のときだけ表示する
<code>state.step < TOTAL_STEPS</code> <code>? A : B</code>	最後のステップより前なら「次へ」、最後なら「この内容で予約する」を表示する
<code>disabled={state.step === 1}</code>	最初のステップでは「戻る」を押せなくする

「この内容で予約する」ボタンはまだ何もしません。送信処理は 11 章で追加します。

BookingForm を配置

`app/shop/[id]/book/page.tsx` に `BookingForm` を配置します。メニューとスタッフは、08 章と同じく `Promise.all` で並列に取得します。

```
import { notFound } from "next/navigation"

import SectionHeading from "@components/SectionHeading"
import { getMenus, getShop, getStaffs } from "@lib/services/shops"
import BookingForm from "@components/booking/BookingForm"

type Props = {
  params: Promise<{ id: string }>
}

export default async function BookingPage({ params }: Props) {
  const { id } = await params

  const shop = await getShop(id)
  if (!shop) {
    notFound()
  }

  const [menus, staffs] = await Promise.all([getMenus(id), getStaffs(id)])

  return (
    <div>
      <SectionHeading
        eyebrow="Booking"
        title={``${shop.name} の予約`}`
        description="メニュー・スタッフ・日時を選んで予約します。"
      />
      <div className="mt-8">
        <BookingForm shop={shop} menus={menus} staffs={staffs} />
      </div>
    </div>
  )
}
```

`page.tsx` は Server Component、`BookingForm` は Client Component です。データの取得はサーバー側で済ませ、取得結果を Props として Client Component に渡します。

動作確認

- 「次へ」「戻る」ボタンで STEP 1～4 が切り替わることを確認します。
- STEP 1 では「戻る」が押せないこと、STEP 4 では「次へ」の代わりに「この内容で予約する」が表示されることを確認します。

BOOKING

nook by linen の予約

メニュー・スタッフ・日時を選んで予約します。

STEP 4: 確認して予約する

戻る

この内容で予約する

まとめ

項目	内容
"use client"	フック・イベントを使うファイルに書く。境界線より下は Client Component として動く
Server / Client の使い分け	データ取得は Server Component、操作する部分だけを Client Component に切り出す
useReducer	<code>dispatch(action)</code> を送り、 <code>reducer</code> 関数が次の state を計算する状態管理フック
useState との使い分け	管理する値・更新パターンが増えるほど useReducer の方が見通しが良くなる
ユニオン型の action	<code>type</code> ごとに一緒に送る値を型で決め、送り間違いをコンパイル時に防ぐ
Context	Provider で囲んだ範囲に値を配り、 <code>useContext</code> で取り出す
useBooking	Context の取り出しと <code>null</code> チェックをまとめたカスタムフック
Provider と利用側の分離	<code>useContext</code> は Provider の内側でしか使えないため、 <code>BookingForm</code> と <code>BookingFlow</code> に分ける
Math.min / Math.max	値が範囲を超えないように制限する
TOTAL_STEPS	ステップ数を定数にして、直し漏れを防ぐ